AetherOS + SpillGuard

A teaching operating system that runs in the browser, connected to a real Windows agent that detects when GPU video memory (VRAM) overflows into system RAM, and a controlled experiment that reproduces, measures and fixes the slowdown this causes.

This document explains **how every part works**, defines **every technical term**, gives the **measured numbers** with their exact meaning, and ends with a **question-and-answer section** and an **A–Z glossary** for the viva.

Student	Varad Patil · Reg. No. 24BIT0175
Institute	VIT · Operating Systems course project
Faculty	Prof. Harshita Patel
Test machine	Laptop with NVIDIA GeForce RTX 3050 6 GB Laptop GPU, AMD Ryzen 7 7840HS, 16 GB DDR5-5600, Windows 11 (build 26200), NVIDIA driver 581.83
Software	Front end: React 18 + TypeScript + Vite + Zustand + Tailwind CSS. Agent: Python 3.12 + FastAPI.

The project in one sentence.
AetherOS *simulates* the classic OS mechanisms (scheduling, paging, deadlock, file-system journaling) so they can be seen; SpillGuard shows the *real* version of the same memory-management problem happening inside a real GPU, and the research part proves with a controlled experiment that crossing VRAM capacity by just 256 MiB makes a local AI model about 2.4× slower.

Contents

1. The big picture

1.1 What was built · 1.2 Why these two belong together · 1.3 Simulated vs real · 1.4 Technology stack

2. Running the demo

2.1 Starting it · 2.2 Suggested demo order · 2.3 What to say in 60 seconds

3. AetherOS architecture

3.1 Layers · 3.2 The kernel clock (tick) · 3.3 Window manager · 3.4 State stores

4. Processes and CPU scheduling

4.1 Process & PCB · 4.2 States · 4.3 Six algorithms · 4.4 MLFQ in detail · 4.5 Dual-core SMP · 4.6 Metrics

5. Memory management

5.1 Paging · 5.2 Page faults · 5.3 FIFO, LRU, CLOCK, OPTIMAL · 5.4 Protection rings & GPF

6. Deadlock

6.1 Coffman conditions · 6.2 Wait-for graph detection · 6.3 Recovery · 6.4 Banker's algorithm

7. File system and crash consistency

8. eBPF Studio, shell, tests and other apps

9. GPU memory fundamentals

9.1 GPU vs CPU · 9.2 VRAM · 9.3 PCIe · 9.4 Shared GPU memory · 9.5 WDDM & VidMm · 9.6 Sysmem fallback · 9.7 Spill

10. Local AI models (why they spill)

11. SpillGuard: the live agent

11.1 Architecture · 11.2 Data sources · 11.3 Detector · 11.4 Classifier · 11.5 Culprit · 11.6 Fit estimator · 11.7 Banker's admission · 11.8 Security · 11.9 Front end

12. SpillGuard evaluation (E1–E5)

13. SpillGuard Repro: the research experiment

13.1 Question · 13.2 Matched block · 13.3 Recipe · 13.4 Verdict rules · 13.5 Safety guards · 13.6 Evidence bundles · 13.7 Reducer

14. Research results (18 Oct 2026)

14.1 Dose-response cliff · 14.2 Same counters, opposite outcomes · 14.3 Remedies · 14.4 Detector check · 14.5 What it does / doesn't show

15. Testing and verification

16. Limitations and future work

17. Viva questions and answers

18. Glossary A–Z

19. Numbers cheat-sheet

1. The big picture

1.1 What was built

The project has three connected parts:

1. **AetherOS** (written "Aethor OS" on the boot screen) — a retro-style desktop operating system that runs entirely inside a web browser. Behind its windows is a small **simulated microkernel** written in TypeScript. It creates processes, schedules them on two simulated CPU cores, gives them memory pages, detects deadlocks, keeps a journaled file system and records kernel events. Every textbook concept is visible and interactive: you can change the scheduling algorithm, change the page-replacement policy, inject a deadlock, cut the power to the disk and run recovery.
2. **SpillGuard** — a **real** monitoring agent written in Python that runs on Windows. Once per second it reads the operating system's own GPU memory counters for every process and detects the moment a program's data stops fitting in the graphics card's memory (VRAM) and starts being placed in ordinary system RAM. This is called a **spill**. The agent serves AetherOS to the browser and streams its measurements into the SpillGuard app, so the simulated OS and the real hardware are shown side by side.
3. **SpillGuard Repro** — a research tool built on the agent. It runs **controlled, repeatable experiments**: a local AI language model (Qwen2.5-3B in Ollama) is timed alone, then with a second "co-tenant" process occupying GPU memory, then alone again. It saves every measurement in a tamper-evident bundle, classifies each trial as FAIL / PASS / INCONCLUSIVE with strict rules, and tests which fixes remove the slowdown.

1.2 Why these two belong together

An OS course teaches **virtual memory**: when the fast memory (RAM) is full, the OS moves some pages to slower storage (disk), and if this happens too often the system **thrashes**. Exactly the same thing happens one level up. A GPU has its own fast memory (VRAM). When programs together need more VRAM than exists, Windows' video memory manager moves some GPU data to slower **system RAM**, reached over the PCIe bus. Nothing crashes — the program just runs much slower, silently. AetherOS teaches the concept with a simulation; SpillGuard measures the real thing on real hardware; the research measures how much it costs and what fixes it.

Textbook concept (simulated in AetherOS)	Real counterpart (measured by SpillGuard)
Physical RAM frames	VRAM on the graphics card (6002 MB on this laptop)
Swap / backing store on disk	"Shared GPU memory" = part of system RAM the GPU can use (7827 MB cap here)
Page fault and page replacement	Windows VidMm evicting/demoting GPU allocations to system memory
Thrashing	Repeated migration of GPU data across PCIe; a 2.4× slowdown measured
Banker's algorithm (deadlock avoidance)	Admission check: "will this new AI model fit in the VRAM that is left?"
Kernel tracing (eBPF)	Live SPILL_START / SPILL_END events in eBPF Studio

1.3 Simulated vs real — be precise about this

SAY THIS CLEARLY IF ASKED

SIMULATED The AetherOS kernel does not control your real CPU or RAM. It is a deterministic model that runs in the browser's JavaScript engine. "Processes", "frames" and "ticks" are data structures, chosen so each algorithm can be observed step by step.

REAL SpillGuard's numbers come from the real Windows performance counters, the real NVIDIA driver and the real GPU. The research results are real measurements on the RTX 3050 laptop.

1.4 Technology stack (and what each word means)

Technology	What it is / why it is used
TypeScript	JavaScript with static types. All of the kernel and UI code is TypeScript, so errors are caught at compile time.
React 18	UI library: the screen is built from components (one per app window) that re-render when their data changes.
Zustand	A small "state store" library. Stores hold shared data (open windows, kernel stats, GPU frames) that any component can read.
Vite	Development server and build tool. `npm run build` type-checks and bundles everything into the `dist/` folder.
Tailwind CSS	Utility CSS classes; plus a custom retro design system in `src/index.css` (classes `rt-window`, `rt-btn`, ...).
react-rnd	Makes windows draggable and resizable.
Vitest / pytest	Test runners for the TypeScript and the Python code.
Python 3.12	Language of the SpillGuard agent and research runner (in a virtual environment `agent/.venv`).
FastAPI + Uvicorn	Python web framework and server. Serves the built AetherOS page and a WebSocket with live data at `127.0.0.1:8765`.
ctypes	Python's way of calling C functions in Windows DLLs directly — used for DXGI and the CUDA driver API, so no C++ helper is needed.
pynvml / psutil	Python bindings for NVIDIA's NVML library; and a library for process names and system memory.
Ollama	A program that runs local AI language models. Its worker process is `llama-server.exe` (built on llama.cpp).

2. Running the demo

2.1 Starting it

1. Double-click `START_DEMO.bat` in the project folder. It builds the app if `dist/` is missing, then starts the SpillGuard agent (`python -m spillguard serve`), which opens the browser at `http://127.0.0.1:8765` with a one-time token.
2. On the AetherOS login screen press **Enter** (it says "It's open. Just press Enter").
3. If the Python environment is missing, the batch file falls back to `npx vite preview` on port 4173 — every OS app and the Replay mode of SpillGuard still work; only the live GPU feed needs the agent.

Developer route: `npm run dev` (port 3000) for the UI with hot reload; `npm test` runs the front-end tests.

2.2 Suggested demo order (≈10 minutes)

1. **Task Manager** — switch between FCFS / SJF / SRTF / Priority / Round Robin / MLFQ; point at the two CPU timelines (Gantt charts), load % and context switches.
2. **Memory Map** — 32 frames of 4 KB; switch LRU / FIFO / CLOCK; spawn a process to cause page faults; click a frame to inspect it.
3. **Deadlock Lab** — press "Inject Deadlock": two threads hold MUTEX_GPU and MUTEX_DISK and wait for each other; the cycle is detected; preempt to recover. Then show the Banker's table and the safe sequence P1 → P3 → P4 → P0 → P2.
4. **Finder** — simulate a power cut, show the dirty superblock and the torn PENDING transaction, run fsck recovery.
5. **Terminal** — `help`, `ps`, `memstat`, `spillguard status`.
6. **SpillGuard** — live VRAM / shared / RAM bars per process; then the **evidence desk**: the headline chart (slowdown vs overflow), remedies, and a live GPU trace replay.

2.3 What to say in 60 seconds

OPENING PITCH

"Operating systems hide memory pressure: when fast memory is full, data is moved to slower memory and programs slow down without any error. AetherOS is a browser OS that makes these mechanisms visible — scheduling on two cores, paging with LRU/FIFO/CLOCK, deadlock detection and Banker's algorithm, and a journaled file system that survives a power cut. SpillGuard applies the same idea to a real modern problem: GPU memory. On Windows, when apps need more VRAM than the card has, the driver silently places the overflow in system RAM over PCIe. My agent reads Windows' per-process GPU counters every second and detects this spill. Then I ran controlled experiments with a local AI model: crossing VRAM capacity by only 256 MiB made it 2.4 times slower, every time, and it recovered fully when the pressure was removed. I also found that the memory counters look identical in a harmful and a harmless case, and that the least disruptive fix is to keep the other program's memory but pause its activity."

3. AetherOS architecture

3.1 Layers

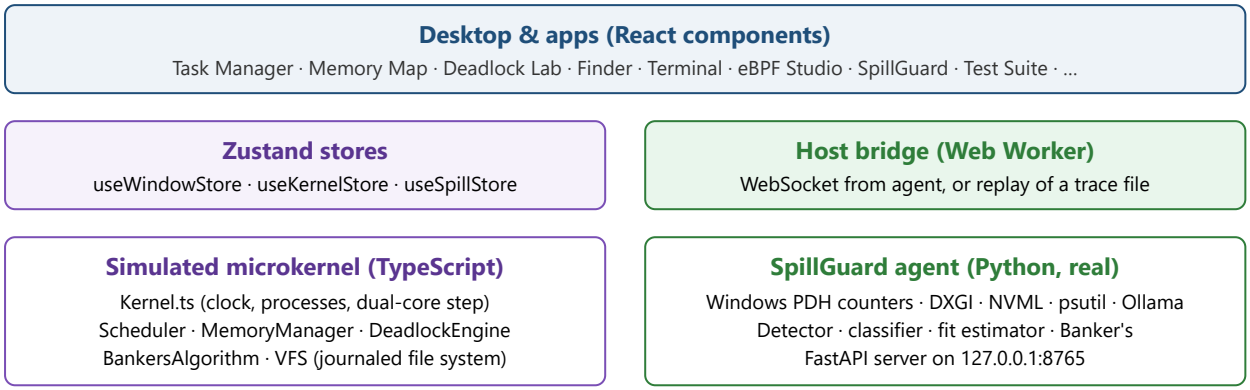


Figure 1. AetherOS layers. Left column is simulated in the browser; right column reads real hardware.

3.2 The kernel clock: a "tick"

Time in the simulated kernel is measured in **ticks**. A JavaScript timer calls `Kernel.step()` every **800 ms** by default (speed is adjustable). One tick = one unit of CPU time. In each tick the kernel:

1. puts whatever ran last tick back to READY, so every tick is a fresh dispatch decision (the policy decides whether the same process continues);
2. synchronizes the per-core run queues with the list of ready processes;
3. does **work stealing** if one core's queue is empty and the other has more than one process;
4. under MLFQ, every 40 ticks applies **aging** (priority boost);
5. asks the scheduler to choose one process for Core 0 and a different one for Core 1;
6. counts a **context switch** whenever a core runs a different process than last tick, and emits an eBPF `SCHED_SWITCH` event;
7. runs each chosen process for one tick (remaining burst −1), touches its memory frames (for LRU/CLOCK), records the Gantt chart entry;
8. marks finished processes TERMINATED, records completion/turnaround time and frees their frames;
9. adds one waiting tick to every process that was READY but not chosen; updates core load percentages.

Because the kernel uses no randomness, the same actions always produce the same result — it is **deterministic**, which makes it testable.

3.3 Window manager

`useWindowStore` keeps one record per app: open/closed, minimized, maximized, position, size and **z-index** (stacking order). Clicking a window "focuses" it by giving it the highest z-index, like a real window manager. Windows are dragged/resized with react-rnd.

3.4 State stores

- **useKernelStore** — wraps the single global kernel object (`globalKernel`) and copies its statistics into React state after every tick (the kernel calls `onUpdate`).
- **useSpillStore** — SpillGuard data: the mode (Simulated / Live / Replay), the latest frame, history, events and peak values.
- **useWindowStore** — windows, as above.

4. Processes and CPU scheduling

4.1 Process and PCB

DEFINITIONS

Process: a program in execution. **PCB (Process Control Block)**: the record the OS keeps for each process. **Burst time**: how much CPU time the process needs. **Arrival time**: when it entered the ready queue.

In AetherOS each process is a `Process` object (the PCB) with these fields:

Field	Meaning
`pid`	Process ID; starts at 100 and increases.
`state`	NEW, READY, RUNNING, WAITING or TERMINATED.
`priority`	1 (highest) to 5 (lowest).
`queueLevel`, `quantumUsed`	MLFQ level (0, 1, 2) and ticks used of the current time slice.
`totalBurstTicks`, `remainingBurstTicks`	CPU work needed in total, and left.
`arrivalTick`, `firstRunTick`, `completionTick`	Timestamps used for the metrics.
`memorySizeKB`, `allocatedFrames`	Memory size and which physical frames hold its pages.
`locksHeld`, `waitingFor`	Mutexes it owns and the one it is blocked on (for deadlock detection).
`coreAffinity`	Which core it may run on: 0, 1 or ANY.

At boot four system processes are created: `kernel_daemon` (pinned to core 0), `window_manager`, `audio_server` (pinned to core 1) and `network_sync`.

4.2 Process states

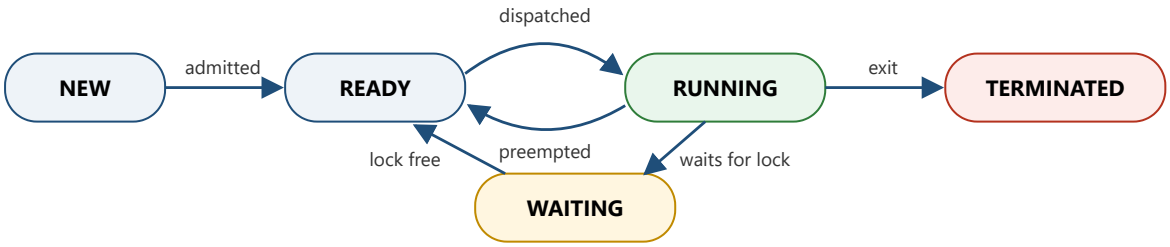


Figure 2. Five-state process model. In AetherOS new processes are admitted straight to READY.

4.3 The six scheduling algorithms

A **scheduler** decides which READY process runs next. **Preemptive** means the OS can take the CPU away from a running process; **non-preemptive** means it runs until it finishes or blocks. Because AetherOS re-decides every tick, the shortest-job and priority policies behave preemptively.

Algorithm	Rule in AetherOS (exact)	Strength / weakness
FCFS First-Come First-Served	Pick the candidate with the smallest `arrivalTick`.	Simple, fair in order; suffers the *convoy effect* (short jobs stuck behind a long one).
SJF Shortest Job First	Pick the smallest `totalBurstTicks`.	Optimal average waiting time if bursts are known; long jobs can *starve*.
SRTF Shortest Remaining Time First	Pick the smallest `remainingBurstTicks` (preemptive SJF).	Even better average waiting; needs burst prediction; starvation.
Priority	Pick the smallest priority number; ties broken by arrival.	Important work first; low priority can starve (fixed by aging).
Round Robin	Run the head of the core's queue; after **time quantum = 4 ticks** move it to the tail.	Fair, good response time; too small a quantum causes many context switches.
MLFQ Multi-Level Feedback Queue	Three queues with quanta **4, 8, 16**; using a full quantum demotes a process one level; every **40 ticks** all are boosted to level 0.	Learns which jobs are interactive vs CPU-bound without knowing burst times. Default algorithm.

Worked example: Round Robin

Two processes A and B, each needing 4 ticks, quantum 2, one core: the schedule is **A A B B A A B B**. A runs 2 ticks (its quantum), is rotated to the tail, B runs 2 ticks, and so on. (The project's built-in test checks exactly this pattern.) With quantum 4 the same pair would run **A A A A B B B B**, which is FCFS — a large quantum turns RR into FCFS.

4.4 MLFQ in detail

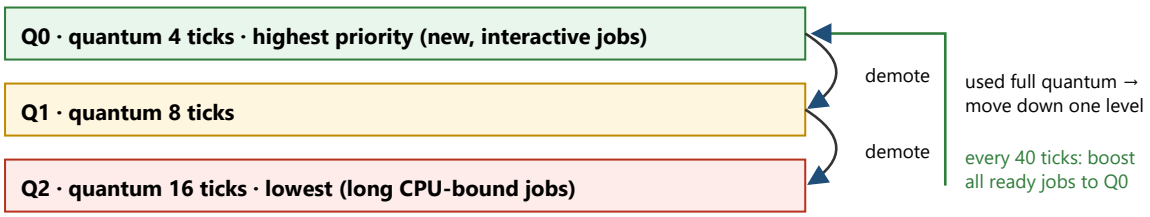


Figure 3. MLFQ: three levels, demotion on quantum exhaustion, periodic aging boost against starvation.

Aging is the cure for **starvation**: a process that waits a long time has its priority raised. In AetherOS, `applyAging()` moves every ready process back to Q0 every 40 ticks and resets its used quantum.

4.5 Dual-core SMP, affinity and work stealing

- **SMP (Symmetric Multi-Processing)**: several identical CPU cores share one memory and one OS. AetherOS simulates two cores, CPU_0 and CPU_1, and steps both every tick; the same process can never run on both cores at once.

- **Run queue**: each core has its own queue of ready PIDs. A new process with affinity ANY is placed on the core with the shorter queue (**load balancing**).
- **Core affinity**: a process can be pinned to core 0 or 1 (like `taskset` on Linux).
- **Work stealing**: if a core's queue is empty and the other has N > 1 processes, the idle core takes ⌊N/2⌋ processes from the *tail* of the other queue (only those whose affinity allows it). This is how Linux's and many runtime schedulers keep cores busy.
- **Gantt chart**: the two coloured timelines in Task Manager show which PID ran on each core in each of the last 30 ticks; grey = IDLE.

4.6 Scheduling metrics (formulas used in the code)

Metric	Formula
Turnaround time	completion tick − arrival tick
Response time	first-run tick − arrival tick
Waiting time	number of ticks the process was READY but not chosen
CPU utilization	busy core-ticks ÷ (2 × elapsed ticks) × 100 %
Throughput	completed processes per 100 ticks
Context switches	count of ticks where a core ran a different PID than the previous tick

A **context switch** is saving the state (registers, program counter) of one process and loading another's; it is pure overhead.

5. Memory management

5.1 Paging

DEFINITIONS

Paging divides a process's memory (its *virtual address space*) into fixed-size **pages** and physical RAM into same-size **frames**. A **page table** maps each page to a frame, so a process's memory need not be contiguous and there is no external fragmentation. **Virtual memory** lets pages that are not needed now live outside RAM.

AetherOS has **32 physical frames of 4 KB** (a 128 KB toy RAM). A process of *S* KB needs ⌈S / 4⌉ pages. The Memory Map app draws the 32 frames as a coloured grid ("frame bitmap"); colour = owning process. Clicking a frame shows its PID, virtual page number, load time and last access time.

5.2 Page faults

A **page fault** happens when a page is needed but not in a frame. If free frames exist, the page is loaded; if not, a **victim** frame must be chosen and evicted — this is **page replacement**. In AetherOS, when a new process needs more frames than are free, each missing frame counts as one page fault and the replacement policy chooses victims (never frames of the same process). Every access to a process's frames (each tick it runs) updates `lastAccessTick` and sets the reference bit.

5.3 Page-replacement policies

Policy	How the victim is chosen in AetherOS	Notes
FIFO	Oldest `loadedAtTick` (loaded first).	Simple, but can evict heavily-used pages; suffers **Belady's anomaly** (more frames can give more faults).
LRU	Oldest `lastAccessTick` (least recently used).	Good approximation of optimal; costly in real hardware. Default policy.
CLOCK (second chance)	A "hand" sweeps the frames in a circle. If the frame's reference bit is 1, clear it and move on (second chance); if 0, evict it. Two sweeps always find a victim.	What real OSes use: approximates LRU with one bit per frame.
OPTIMAL (Bélády's MIN)	Needs the future reference string, which the kernel does not model, so it falls back to LRU (the UI labels this honestly).	Evict the page used farthest in the future; impossible online; used as a benchmark.

Worked example (3 frames, reference string 7 0 1 2 0 3 0 4)

Policy	Frames after each reference (F = fault)	Faults
FIFO	7F · 70F · 701F · 012F · hit · 123F · 230F · 304F	7
LRU	7F · 70F · 701F · 012F (evict 7) · hit · 023F (evict 1) · hit · 034F (evict 2)	6
OPTIMAL	evict the page not needed for longest: 7, then 1, then 2	6

5.4 Protection rings and the General Protection Fault

CPUs have privilege levels called **rings**. **Ring 0** = kernel mode (full access); **Ring 3** = user mode (restricted). This is **dual-mode operation**: user programs cannot touch kernel memory. AetherOS's toy address space is 64 KB: **0x0000–0x3FFF** (16 KB) is kernel space, **0x4000–0xFFFF** is user space.

The "trigger GPF" action makes a Ring-3 process read address **0x0020** (inside kernel space). The memory manager's `verifyAddressAccess` traps it as a **General Protection Fault (#GP)**, moves the process to WAITING (`SIGSEGV_FAULT_HANDLER`) and shows a **register dump**:

Register	Meaning (x86)
EIP	Instruction pointer — address of the faulting instruction.
ESP	Stack pointer — top of the process's stack.
CR3	Control register 3 — physical address of the current page table (the address space).
EAX	General-purpose accumulator register (here holds the illegal address).
CS	Code segment selector; 0x0073 is a user-mode (ring 3) code segment.
EFLAGS	Status flags (zero, carry, interrupt-enable …).

The values in the dump are generated for illustration; the mechanism (check ring + address range → trap) is the real concept.

6. Deadlock

6.1 Definition and the four Coffman conditions

A **deadlock** is a set of processes each waiting for a resource held by another process in the set, so none can proceed. It can occur only if all four conditions hold at once:

1. **Mutual exclusion** — a resource can be held by only one process (e.g. a mutex).
2. **Hold and wait** — a process holds one resource while waiting for another.
3. **No preemption** — resources cannot be forcibly taken away.
4. **Circular wait** — a cycle P1 → P2 → … → P1 of "waits for".

Strategies: **prevention** (break a condition), **avoidance** (Banker's algorithm), **detection and recovery** (find the cycle, then kill or preempt).

6.2 Detection with a wait-for graph

AetherOS's resources: `MUTEX_GPU`, `MUTEX_DISK`, `MUTEX_NET` (1 instance each) and `SEMAPHORE_IO` (2). "Inject Deadlock" creates `Worker_Thread_Alpha` (holds MUTEX_GPU, waits for MUTEX_DISK) and `Worker_Thread_Beta` (holds MUTEX_DISK, waits for MUTEX_GPU).

`DeadlockEngine.evaluateDeadlock` builds an **allocation matrix** (who holds what) and a **request matrix** (who waits for what), then a **wait-for graph**: an edge P → Q means "P waits for a resource Q holds". It runs a **depth-first search (DFS)** keeping a *recursion stack*; meeting a node already on the stack means a **back edge**, i.e. a cycle. The PIDs on the cycle are reported. Complexity is O(V + E).

> **Mutex** = mutual-exclusion lock (one holder). **Semaphore** = counter allowing up to N holders. For single-instance resources a cycle in the wait-for graph is necessary *and sufficient* for deadlock; with multiple instances a cycle is necessary but not sufficient.

6.3 Recovery

The Deadlock Lab offers **"Preempt offending cycle processes"**: terminating a process on the cycle releases its locks and breaks the circular wait.

6.4 Banker's algorithm (deadlock avoidance)

Dijkstra's Banker's algorithm grants a request only if the system stays in a **safe state** — a state with at least one order (**safe sequence**) in which every process can obtain its maximum claim and finish.

- **Available[j]**: free instances of resource j. **Max[i][j]**: the most process i will ever need. **Allocation[i][j]**: what i holds now.
- **Need = Max − Allocation**.
- Safety algorithm: Work = Available. Repeatedly find an unfinished process with Need ≤ Work; pretend it finishes and returns its allocation (Work += Allocation[i]). If all finish, the state is safe.

The AetherOS example (resources A, B, C; Available = [3, 3, 2])

Process	Allocation	Max	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Trace: Work 3 3 2 → P1 runs (need 1 2 2) → Work 5 3 2 → P3 → 7 4 3 → P4 → 7 4 5 → P0 → 7 5 5 → P2 → 10 5 7. **Safe sequence: P1 → P3 → P4 → P0 → P2.** (This is the textbook Silberschatz example.)

SpillGuard re-uses the very same `runBankersSafetyCheck` function on **real VRAM numbers** — see section 11.7.

7. File system and crash consistency

7.1 Inodes and the superblock

An **inode** (index node) stores a file's metadata: number, name, type (file/directory), size, permissions (`rw-r--r--`) and parent. The **superblock** describes the whole file system: whether it is *clean*, *mounted*, inode count, block size (256 bytes here) and journal head/tail. AetherOS starts with `/`, `/user`, `/etc`, `readme.txt` and `manifest.json`.

7.2 Write-ahead logging (journaling)

A crash in the middle of an update can leave metadata half-written ("torn"). **Write-ahead logging (WAL)** fixes this: first write *what you intend to do* to a log (journal), then do it. Every create/delete in AetherOS goes through three states:

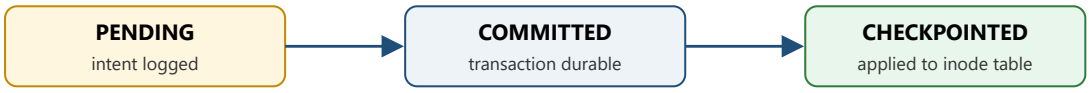


Figure 4. Journal entry life cycle (ring buffer of 25 entries).

7.3 Power cut and fsck recovery

- **Power cut** (`triggerPowerCut`): marks the superblock dirty, unmounts the disk and leaves a torn `WRITE` to `/user/dirty_buffer.tmp` in state PENDING. While unmounted, file operations fail.
- **fsck** (file-system check, `recoverFSCK`): scans the journal; **rolls back** every PENDING (never committed) entry and **replays** every COMMITTED but not checkpointed entry; then marks the superblock clean and remounts.

This is the same idea as ext4's journal, NTFS's log file and database write-ahead logs: committed work survives a crash, uncommitted work disappears atomically.

8. eBPF Studio, shell, tests and other apps

8.1 eBPF Studio (kernel tracing)

eBPF (extended Berkeley Packet Filter) is a real Linux technology: small verified programs attached to kernel *hooks* (tracepoints) run when an event happens and store counts in **BPF maps** (key–value tables), or stream events through a **ring buffer**. AetherOS simulates this: three probes are attached at boot —

- `sched_switch_counter` on **SCHED_SWITCH** (every context switch, counted per PID);
- `mm_page_fault_tracer` on **PAGE_FAULT** (memory mapping and GPF traps);
- `vfs_syscall_latency` on **SYSCALL_ENTER**.

You can attach, toggle and remove probes; each shows its map (`BPF_MAP_TYPE_HASH`) and the last 40 events. SpillGuard's SPILL_START / SPILL_END events also appear here. (The filter script text is a label; it is not compiled to real BPF bytecode.)

8.2 Terminal (aethersh)

A command-line shell that talks to the simulated kernel:

Command	Effect
`ps` · `kill <pid>` · `run <name>`	List processes, terminate one, spawn one.
`sched <algo>` · `quantum <n>` · `affinity`	Change the scheduling algorithm, the RR time quantum, a process's core affinity.
`memstat` · `gpf`	Memory/page-fault statistics; trigger the protection-fault demo.
`powercut` · `fsck`	Simulate the power cut; run journal recovery.
`bpf`	Show eBPF probes and counts.
`pwd ls cd cat touch mkdir rm`	File-system commands on the journaled VFS.
`open <app>` · `neofetch` · `uname` · `uptime` · `clear`	Desktop and system-info commands.

SpillGuard commands:

```
spillguard status            VRAM / shared / RAM tiers and per-process state
spillguard why <pid>         explain one process's spill state and its culprit
spillguard events            recent SPILL_START / SPILL_END / OFFLOAD_SPLIT events
spillguard fit <model> [ctx] [f16|q8_0|q4_0]   VRAM estimate + Banker's admission
spillguard mode <simulated|live|replay>
spillguard demo              replay the recorded RTX 3050 spill trace
```

For safety the browser never launches programs: `spillguard run` only prints the command to use in a real terminal.

8.3 Kernel Test Suite, Audit Journal, KernelQuest

- **Kernel Test Suite** — runs unit, integration and end-to-end checks inside the OS: FCFS order, Round-Robin quantum, SRTF, MLFQ aging, LRU and FIFO eviction, an unsafe Banker's state, deadlock-cycle detection, inodes, terminal integration and state export/import.
- **Kernel Audit Journal & Invariants** — the kernel's event log (BOOT, PROCESS_CREATE, WORK_STEAL, AGING, DEADLOCK, GPF_FAULT …, last 50).
- **KernelQuest** — guided scenarios ("quests") that ask the user to trigger and resolve OS situations; quest 3 ("Who is the culprit?") uses a real GPU trace and quest 4 ("Will this model fit?") asks the user to compute M = W + KV + B + O by hand.

8.4 Other desktop apps

Finder (file explorer), Calculator, Music player, Messages, a simulated Internet browser, Community page, Settings (wallpaper and desktop), Trash, About/ReadMe, and the desktop pet bird (**Zephyr**) in the corner, which gives tips and announces SpillGuard spill alerts. These make it feel like a real desktop; the OS concepts live in the apps described above.

9. GPU memory fundamentals

9.1 GPU vs CPU

A **CPU** has a few powerful cores optimized for sequential, branchy code. A **GPU** (Graphics Processing Unit) has thousands of small cores that run the same operation on many data items at once (**SIMD/SIMT** parallelism). That is ideal for graphics and for the matrix multiplications inside AI models. A GPU computes fast only if it can be fed data fast — so its memory bandwidth matters most.

9.2 What is VRAM?

DEFINITION

VRAM (Video RAM) is the GPU's own dedicated memory, physically on the graphics card (or next to the GPU chip on a laptop board). On this laptop it is **6 GB of GDDR6** (Windows reports 6002 MB as `DedicatedVideoMemory`). GDDR6 = Graphics Double Data Rate 6, memory designed for very high bandwidth.

Bandwidth = how many bytes per second can be moved. Measured on this GPU (experiment E1): writing to data in VRAM ran at **163.3 GB/s**; writing to GPU data that the driver had placed in system RAM ran at **13.2 GB/s — 12.4 times slower**.

Units: 1 MB = 10^6 bytes; **1 MiB (mebibyte) = 2^{20} = 1,048,576 bytes**; 1 GiB = 2^{30} bytes. The research uses MiB; Windows counters report whole MB/MiB.

9.3 PCIe — the road between GPU and system RAM

PCIe (PCI Express) is the serial bus connecting the GPU to the CPU and system RAM. The GPU can read system RAM across it, but far more slowly than its own VRAM. That slow road is why spilling costs so much.

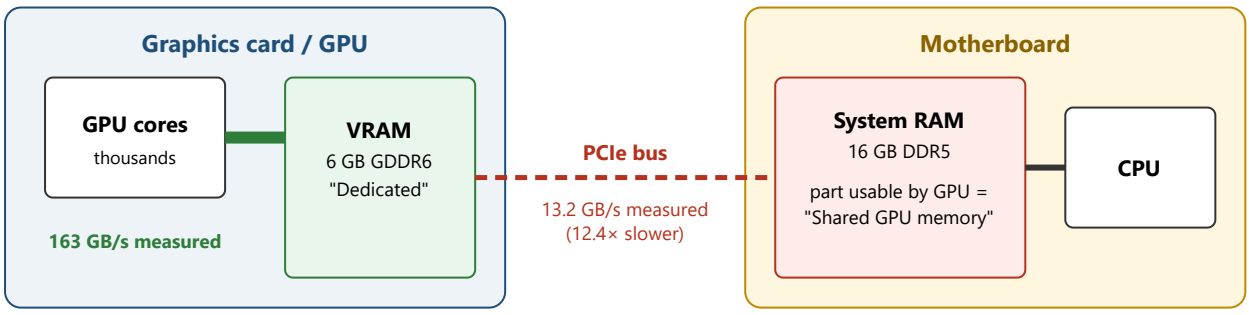


Figure 5. Two memory tiers for GPU data. Bandwidths are this project's measurements (experiment E1).

9.4 "Shared GPU memory"

Windows lets a GPU use part of system RAM as a second memory tier. Task Manager shows it as **Shared GPU memory**. Its limit is half of system RAM (Microsoft's documented rule): **7827 MB** on this 16 GB laptop. Data in this tier is in normal DDR5 RAM and the GPU reaches it over PCIe.

9.5 WDDM and VidMm — the "operating system" of GPU memory

- **WDDM (Windows Display Driver Model)**: the architecture for graphics drivers on Windows. GPU memory is managed by Windows, not only by the vendor driver.
- **VidMm (Video Memory Manager)**: the part of the Windows graphics kernel (`dxgkrnl.sys`) that decides where every GPU allocation lives. It gives GPUs **virtual memory**: allocations get GPU virtual addresses, and VidMm can make them **resident** in VRAM or move (**evict / demote**) them to system memory when VRAM is over-committed — exactly like an OS paging RAM to disk.
- **Residency**: whether an allocation is currently in VRAM (fast) or not.
- **Budget**: the amount of VRAM Windows suggests one process should use, queried with DXGI's `QueryVideoMemoryInfo`. It is **per calling process**. On this laptop the budget was 5234 MB, yet under pressure the driver filled physical VRAM to 5954 of 6002 MB — so SpillGuard measures pressure against **physical** VRAM, not the budget.
- **DXGI (DirectX Graphics Infrastructure)**: the Windows API that lists graphics adapters and reports their memory.
- **LUID (Locally Unique Identifier)**: the ID Windows gives each adapter; counter names contain it, e.g. `pid_32220_luid_0x00000000_0x00018056_phys_0`.

9.6 NVIDIA "sysmem fallback"

Since driver version **536.40**, when a CUDA program asks for more VRAM than is free, the NVIDIA driver on Windows does not fail with "out of memory"; it **falls back to system memory** (setting "CUDA – Sysmem Fallback Policy" in the NVIDIA Control Panel). The program keeps running, but slower. Project finding: `cuMemGetInfo` (CUDA's "how much memory is free?" call) reported **0 MB free from 5376 MB onward, and allocations kept succeeding** — CUDA itself gives no signal that it is spilling. That is why an external detector is needed.

9.7 So what exactly is a "spill"?

DEFINITION USED IN THIS PROJECT

A **spill** is when part of a process's GPU data that would normally live in VRAM is placed in system memory ("shared GPU memory") because VRAM is full. It shows up as the process's **Shared Usage** counter growing while the adapter's dedicated VRAM is near its physical limit. The program does not crash; memory accesses to the spilled part go across PCIe and the program slows down.

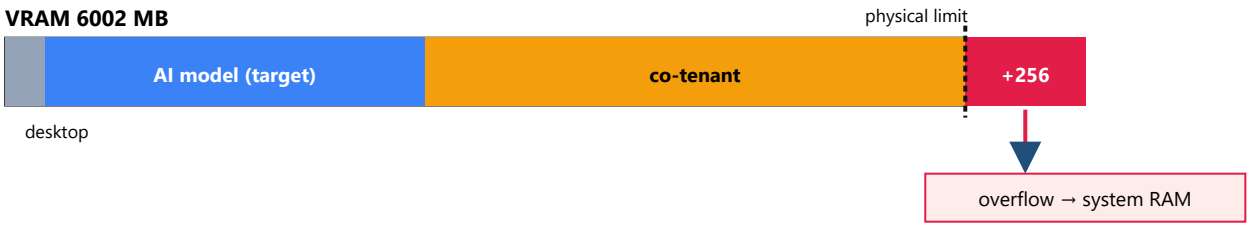


Figure 6. A spill: the sum of everyone's VRAM demand passes the physical limit; the excess is placed in system RAM.

10. Local AI models — why they spill

The research target is a **large language model (LLM)** running locally. Key terms:

- **Parameters / weights**: the numbers learned in training. Qwen2.5-3B has about 3 billion.
- **Quantization**: storing weights with fewer bits (e.g. 4-bit instead of 16-bit) to save memory, with small accuracy loss. Formats: `f16` (2 bytes), `q8_0` (≈1.06 bytes), `q4_0` (≈0.56 bytes) per value.
- **GGUF**: the model file format of llama.cpp / Ollama; its header lists every tensor with its size and type, which SpillGuard reads to estimate memory.
- **Layers**: a transformer is a stack of identical blocks (36 for Qwen2.5-3B). **Offloading** = putting layers on the GPU; `num_gpu` / `-ngl` chooses how many. If not all fit, the framework splits the model between GPU and CPU ("partial offload", called type A in SpillGuard).
- **Token**: a piece of a word; the model generates one token at a time.
- **Prefill** (prompt processing) reads the whole prompt in parallel; **decode** generates output tokens one by one. Decode reads all weights for every token, so it is limited by memory bandwidth — and therefore very sensitive to spill. The research metric is **decode milliseconds per token** (lower = faster).
- **Context length** (`num_ctx`): how many tokens the model can attend to. The research uses 2048.
- **KV cache**: memory that stores the attention *keys* and *values* of past tokens so they are not recomputed. Size grows with context.
- **Temperature 0 and seed 0**: deterministic decoding, so the output text must be identical in every run — the research checks this with a hash.
- **Ollama**: a local model server; its worker is `llama-server.exe`. **llama.cpp**: the C++ inference engine underneath.

KV-cache formula (used in SpillGuard's fit estimator)

```
KV bytes = 2 (keys and values) × layers × n_ctx × n_head_kv × head_dim × bytes_per_element
Qwen2.5-3B, ctx 2048, f16:  2 × 36 × 2048 × 2 × 128 × 2 bytes = 75,497,472 bytes ≈ 72 MiB
```

Measured total GPU footprint of the target in the research setup (weights + KV + CUDA context + work buffers): about **2421 MiB**.

11. SpillGuard: the live agent

11.1 Architecture

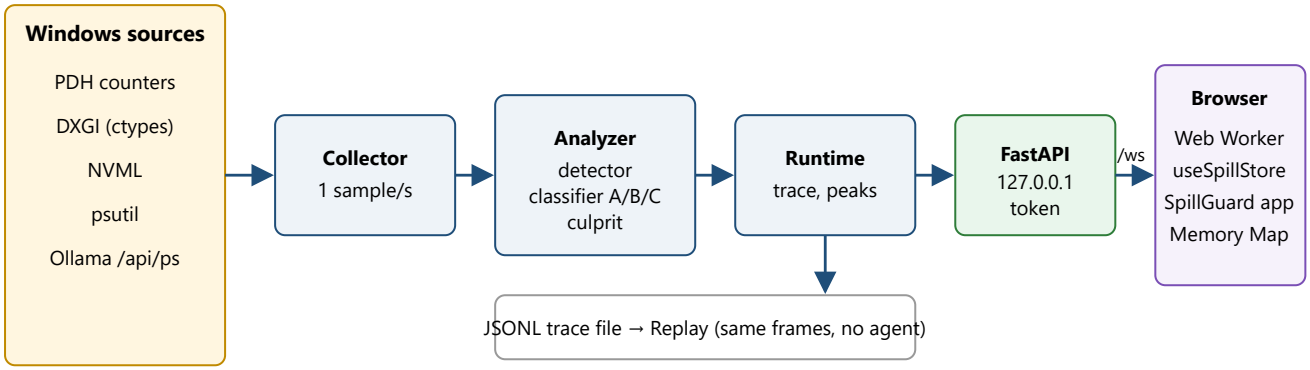


Figure 7. SpillGuard data flow. One JSON "frame" per second travels from Windows counters to the AetherOS window.

11.2 Data sources (what each one is)

Source	What it is	What SpillGuard reads
PDH	Performance Data Helper — the Windows API for performance counters (the same data Task Manager shows). No administrator rights needed.	Per process: `\GPU Process Memory(*)\Dedicated Usage` and `Shared Usage`. Per adapter: `\GPU Adapter Memory(*)\Dedicated Usage`, `Shared Usage`. Instance names are enumerated at run time, never hand-built.
DXGI	DirectX Graphics Infrastructure, called through ctypes vtables.	Adapter list, physical VRAM (`DedicatedVideoMemory`), the OS budget (`QueryVideoMemoryInfo`).
NVML	NVIDIA Management Library (what `nvidia-smi` uses).	Total used/free VRAM, temperature, clocks, power, utilization — a cross-check of PDH.
psutil	Python system library.	Process names, commit charge %, available RAM.
Ollama /api/ps	Ollama's HTTP endpoint listing loaded models.	`size` vs `size_vram`: whether Ollama itself split the model between CPU and GPU.

Dedicated Usage = bytes of a process's GPU allocations resident in VRAM. **Shared Usage** = bytes in system memory. **Commit charge** = total virtual memory Windows has promised to all processes (RAM + page file); near 100 % the system is close to running out.

11.3 The spill detector (type B) — exact algorithm

Run once per second for every process, with **hysteresis** (different thresholds for entering and leaving a state, so it does not flicker):

Parameter	Default	Meaning
θ_{hi} (theta high)	0.92	Adapter "under pressure" when dedicated VRAM ≥ 92 % of physical VRAM.
θ_{lo} (theta low)	0.85	Pressure ends below 85 %.
Δ (delta)	128 MB	Shared-usage growth above the process's own baseline that counts as spill.
k	2	Consecutive positive samples needed to enter SPILLING.
exit_k	3	Consecutive clear samples needed to leave SPILLING.

1. **Baseline**: for each process, the *minimum* shared usage seen while the adapter was *not* under pressure (last 60 samples). Browsers and the compositor (`dwm.exe`) always keep some shared memory; the baseline stops them from being flagged.
2. **Excess** = current shared usage − baseline.
3. **Enter SPILLING** when for k = 2 samples in a row: load ≥ θ_{hi} *and* excess > Δ *and* shared usage is still rising (5-sample slope window). Emit **SPILL_START**.
4. **Leave** after 3 clear samples (load < θ_{lo} or excess ≤ Δ/2). Emit **SPILL_END** with the duration.

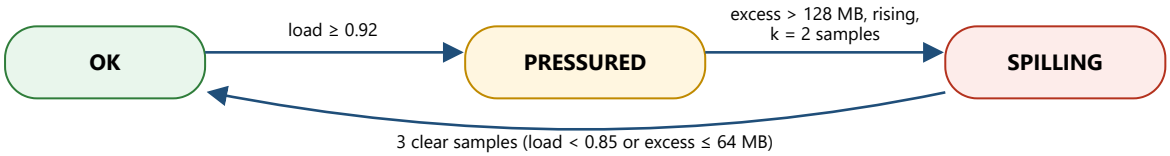


Figure 8. Per-process detector state (simplified).

The analyzer is **pure and deterministic**: feeding it a recorded trace gives exactly the same events as live, which is how it is unit-tested and how thresholds were tuned.

11.4 Cause classifier A / B / C

Type	Meaning	Signal
A — framework split	The AI framework chose to keep some layers on the CPU (not a driver spill).	Ollama reports `size_vram < size` → event OFFLOAD_SPLIT.
B — driver spill	The driver/VidMm placed GPU data in system RAM.	The detector above fires.
C — host RAM pressure	The whole computer is short of RAM.	Commit ≥ 90 % or available RAM < 512 MB for 3 samples. B during C is reported "B+C".

11.5 Culprit

The process whose dedicated VRAM grew most in the **30 seconds before** the spill started. If it is the spilling process itself the spill is "self-inflicted"; otherwise the spiller is a **victim** of another app. Example message from a live test: "SPILLING: python.exe (1.50 GB, 21 %)… python.exe itself grew VRAM by 3.00 GB in the 30 s before the spill".

11.6 Fit estimator: will this model fit?

```
M = W_gpu + KV + B + O
W_gpu = bytes of the weight tensors of the layers placed on the GPU (llama.cpp offloads the LAST
n_gpu_layers),
        + output head (+ a GPU copy of token_embd when embeddings are tied) + vision encoder if any
KV    = 2 · layers · n_ctx · n_head_kv · head_dim · bytes/elem   (f16 = 2, q8_0 = 34/32, q4_0 =
18/32)
B     = logits buffer + attention scratch (only without flash attention) + calibrated terms
O     = fixed overhead (CUDA context etc.)
```

Calibrated on this laptop: **O = 235 MB, +6.1 MB per 1 k context, +535 MB image-encoder buffer for vision models**. Accuracy (E3): leave-one-out mean absolute error **46 MB** (max 93 MB) over 9 configurations; Ollama's own estimate was off by **902 MB** on average because it leaves out the CUDA context and buffers.

11.7 Banker's algorithm as GPU admission control

Before launching a new model, SpillGuard asks the Banker's question with **VRAM (and commit) as the resources**: Allocation = each running process's measured VRAM; Max = its observed peak × 1.1; the new launch has Max = its estimated need; Available = physical VRAM − 256 MB margin − total allocation.

Important twist: **real applications are long-lived** — a game or a browser never "finishes" and returns its memory, which the textbook algorithm assumes. So SpillGuard computes two verdicts and shows both in the Deadlock Lab: the **classic** safe-sequence result and the **long-lived** result, which reserves every running process's remaining need up front. Verdicts: **SAFE**, **FIXABLE** (it searches context length, KV-cache type, flash attention and GPU layers for the best configuration that fits) or **UNSAFE** with a plain reason ("needs 6.5 GB VRAM, 5.4 GB free; chrome.exe holds 1.4 GB"). `spillguard run -- <command>` can rewrite llama.cpp flags (`-c -ngl -fa -ctk -ctv`) or derive an Ollama model with `num_ctx` / `num_gpu` , and always asks before starting.

11.8 Security model

- Binds to **127.0.0.1** only (loopback) — not reachable from the network.
- A random **session token** per run is required for `/ws` and `/api/*` . It travels in the URL **fragment** (`#sg=…`), which browsers never send to servers, and the page removes it from the address bar.
- **Host** and **Origin** headers are checked, blocking other websites and **DNS rebinding** (an attack where a malicious domain is re-pointed to 127.0.0.1). Verified: no token → 401, foreign Origin → 403, rebinding Host → 403.
- The HTTP API is **read-only**. Launching programs is CLI-only behind a confirmation. Driver settings are never changed automatically.

11.9 Front end: three modes

- **Simulated** — synthetic data, no agent needed.
- **Live** — a **Web Worker** (a background browser thread) holds the WebSocket and parses frames off the UI thread.
- **Replay** — plays a saved **JSONL** trace (one JSON object per line: a header with machine facts and detector parameters, then one frame per second). Replay uses exactly the same frames as live, so the demo works on any computer.

Views: stacked tier bars (VRAM / shared / RAM), per-process table with state OK / PRESSURED / SPILLING, event list, the admission panel (slider for a new launch's VRAM claim), the fit form (model, context, KV type), the **VRAM twin view** inside the Memory Map, and the research **evidence desk** (section 14).

12. SpillGuard evaluation (E1–E5)

Ground truth must not come from the detector's own input. The workload `bench/spillstress.py` (called W1) allocates GPU memory in 256 MB blocks through the CUDA *driver* API (`nvcuda.dll`), warms the GPU out of its idle power state, then measures the write bandwidth of each block. Blocks the driver placed in system RAM are an order of magnitude slower — that marks the true spill onset.

Experiment	Setup	Result
E1 cost of spill	W1, 3 runs, per-block write bandwidth	Resident **163.3 ± 0.1 GB/s**, spilled **13.2 ± 0.0 GB/s** → **12.4× slower**. Spill always began at the 23rd 256 MB block (5.75 GB allocated).
E1b type A	qwen3-vl:2b, 32 k context	Ollama split the model itself (3.6 of 5.5 GB on GPU): **71 vs 113 tokens/s**.
E2 detection	3 spill runs + 2 runs reaching ~98 % VRAM *without* spilling	Recall **3/3**, **0** false positives, latency **0.95 ± 0.03 s** after true onset.
E3 fit estimator	2 models × 5 contexts in Ollama	Leave-one-out MAE **46 MB** (max 93); Ollama's own estimate off by **902 MB**.
E5 overhead	agent self-measurement at 1 Hz	**2.3 ± 0.3 %** of one core ≈ 0.14 % of the 16-thread CPU; **52 MB** RAM.

Recall = detected spills ÷ real spills. **Precision** = real spills ÷ all alarms. **False positive** = an alarm with no real spill. **Latency** = time from true onset to the alarm. **MAE** (mean absolute error) = average |estimate − measured|. **Leave-one-out** = each configuration is predicted by a model fitted without it (an honest test of prediction). **± value** = standard deviation over repeated runs.

13. SpillGuard Repro: the research experiment

13.1 Research question

Title of the research direction: *"SpillGuard Repro: Reproducing and Minimizing Cross-Process GPU-Memory Performance Failures on Windows."* The questions: **(RQ1)** can a severe slowdown of a *real* application, caused by *another* process's GPU memory use, be reproduced repeatably and recover when the pressure is removed? **(RQ2)** can the pressure be reduced to a minimal cause? **(RQ3)** which remedy fixes it with the least disruption? A failure caused by a neighbour process is called a **cross-process** failure; the neighbour is the **co-tenant**.

13.2 The matched block (baseline → pressure → recovery)

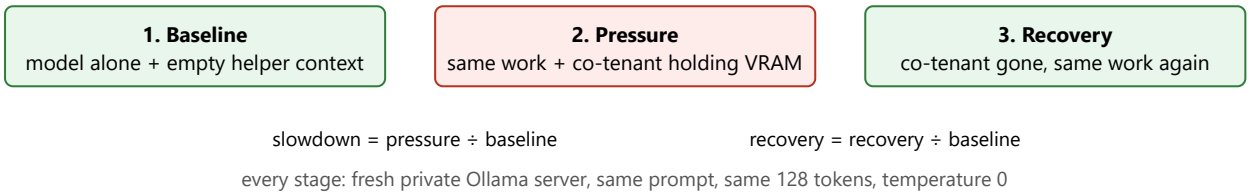


Figure 9. One matched trial. Comparing the target against itself under identical conditions isolates the effect of the co-tenant.

Why this design: comparing the same work, on the same machine, minutes apart, with only one thing changed is a **controlled experiment**. The recovery stage proves the slowdown was *caused* by the pressure (it goes away when the pressure goes away) and not by heat, a background update, or a broken run. All three stages contain an empty helper CUDA context, so the extra context itself is not the difference.

13.3 The recipe (experiment specification)

Every experiment is a JSON **recipe** validated before running (unknown keys, non-finite numbers, duplicate IDs and out-of-bound sizes are rejected).

Item	Value in the 18 Oct study
Target workload	Ollama 0.34.4, model `qwen2.5:3b-instruct` (digest pinned), fixed prompt (SHA-256 pinned), `num_predict` 128, `num_ctx` 2048, temperature 0, seed 0, `num_gpu` 999 (all layers on GPU)
Primary metric	Server-reported **decode ms per token**
Co-tenant ("holder")	An owned Python process allocating CUDA memory via `nvcuda.dll`; size = **physical VRAM − desktop residency − target footprint (2421 MiB) + planned overflow**
Activity	Touches (rewrites) all its memory every **250 ms** ("active"), or once ("idle", touch interval 0)
Preparation	**cold**: the co-tenant becomes resident *before* the model loads (pressure-phase anchor, 4 s lead-in)
Telemetry	PDH sampling every 100 ms during each stage
Repeats	2 matched trials per level

13.4 Verdict rules (the classifier in `evidence.py`)

Each trial gets exactly one of three verdicts:

- **FAIL** = slowdown reproduced: pressure ÷ baseline ≥ **2.0** *and* recovery ÷ baseline ≤ **1.25**.
- **PASS** = valid non-reproduction: a clean trial whose slowdown stayed below 2.0.
- **INCONCLUSIVE** = anything doubtful, never silently counted as PASS or FAIL. Including: a stage failed or timed out; the requested work, prompt, model identity, realized context or **generated output** changed; host-memory guard tripped; Ollama reported partial CPU offload (that would be type A, not a cross-process spill); the pressure was not observed during the target (fewer than 2 overlapping samples); recovery did not return to baseline.

A whole block of repeats is FAIL or PASS only if all its trials agree; mixed outcomes are INCONCLUSIVE. The 2× threshold is an **operational definition** chosen before the experiment, not a hardware constant. "FAIL" names the *application's* failure to keep its performance, not a failure of the experiment.

13.5 Safety guards

- Abort the stage if available host RAM falls below the floor (**1.5 GiB** in this recipe) or **commit ≥ 90 %** — a trial run while the whole PC is short of RAM would measure host pressure, not GPU spill (type C). Such trials are kept and marked INCONCLUSIVE. These guards were **never relaxed** to get results; memory was freed by closing apps.
- Only processes the runner itself started can be stopped; no third-party process is killed, no driver setting is changed.
- Requested pressure is capped (default 1 GiB, max 8 GiB explicitly declared).

13.6 Evidence bundles

Each run is saved as a **bundle**: `recipe.json`, `report.json`, readable `report.md`, per-stage helper logs, observations and `trace.jsonl` files (replayable in AetherOS), plus a **SHA-256 manifest**. SHA-256 is a cryptographic hash: changing even one byte of a file changes its hash, so `verify` detects any modification. Invalid trials stay in the record and in the denominator — nothing is cherry-picked. The 19 files shown in AetherOS's evidence desk were copied with the hostname removed and unrelated process names replaced by "other app".

13.7 The reducer (delta debugging)

ddmin (delta debugging minimization, Zeller & Hildebrandt, 2002) shrinks a failing input by deleting parts and re-testing, keeping a deletion only if the failure still reproduces. Its result is **1-minimal**: removing any single remaining part makes the failure disappear. SpillGuard Repro implements plain ddmin over pressure *actions*, with a test budget and a fresh final validation. It is validated on a labelled software fixture (4 actions reduced to 2). Applying a phase/residency-aware reducer to the real witness is **future work (RQ2)**.

14. Research results (18 October 2026)

14.1 Result 1 — a performance cliff at the capacity boundary

"Planned overflow" = how much the co-tenant plus the model plus the desktop exceed physical VRAM. Negative = fits.

Planned overflow	Co-tenant holds	Baseline ms/tok	Pressure ms/tok	Slowdown	Verdict
−512 MiB (fits)	2,941 MiB	14.4 / 14.3	16.6 / 16.7	1.15× / 1.17×	PASS / PASS
0 MiB (at capacity)	3,453 MiB	14.2 / 14.0	17.1 / 17.1	1.20× / 1.22×	PASS / PASS
+256 MiB	**3,709 MiB**	**14.0 / 14.1**	**33.3 / 33.1**	**2.37× / 2.35×**	**FAIL / FAIL**
+512 MiB	**3,965 MiB**	**14.1 / 14.1**	**33.6 / 32.2**	**2.38× / 2.28×**	**FAIL / FAIL**

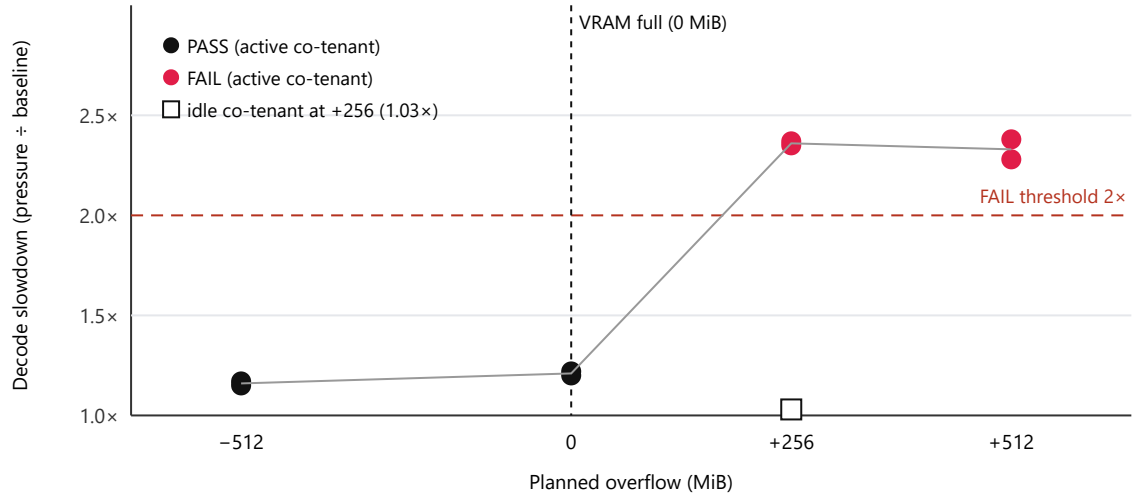


Figure 10. Dose-response: slowdown stays ~1.2× up to capacity, then jumps to ~2.35× just 256 MiB past it, and does not get worse at +512 MiB.

- **Dose-response** means measuring the effect at several doses (here: amounts of overflow) — like testing a medicine at several strengths.
- The response is a **cliff with a plateau**: +256 MiB more than doubles per-token time; +512 MiB is no worse.
- The −512 MiB control (the co-tenant rewrites ~2.9 GiB every 250 ms but everything fits) gives only 1.16×: the co-tenant's own bandwidth use explains a small slowdown, not 2.36×.
- Every FAIL recovered on release (recovery 0.996–1.010×) with byte-identical output and no partial CPU offload.

14.2 Result 2 — identical memory counters, opposite outcomes

Co-tenant at +256 MiB	Runner VRAM / shared (PDH)	Co-tenant VRAM	Adapter VRAM	Slowdown
active (touch every 250 ms)	2,033 / 388 MiB	3,790 MiB	5,952 MiB (ceiling)	**2.37× (FAIL)**
idle (touch once)	2,033 / 388 MiB	3,790 MiB	5,952 MiB (ceiling)	**1.03× (PASS)**

The memory counters are **the same** in both cases, yet one is a severe slowdown and the other is not. So **counters alone cannot tell harmful from benign pressure**; an **intervention** (changing one thing and re-measuring) can. In this run the idle case had one valid trial (two were stopped by the commit guard); the "pause activity" remedy below adds two more valid idle trials (1.01×, 1.01×).

> **Working hypothesis, not verified:** an *active* co-tenant makes VidMm keep migrating pages between VRAM and system RAM ("churn"), which 100 ms counter snapshots cannot see. Proving it needs PIX / ETW residency event traces (future work). Say "hypothesis" — not "proved".

14.3 Result 3 — the witness re-validates and every remedy works

A **witness** is a saved recipe that reproduces the failure. The +256 MiB witness was re-run from scratch, then each remedy as its own matched block (2 trials each).

Remedy	Co-tenant still holds	Slowdown per trial	Verdict
none (fresh re-validation)	3,709 MiB	2.35×, 2.34×	FAIL (reproduced again)
release the whole co-tenant	0 MiB	1.02×, 1.00×	PASS
run the co-tenant after the target (serialize)	0 MiB	1.00×, 1.00×	PASS
release half of its bytes	1,854 MiB	1.10×, 1.10×	PASS
release one eighth of its bytes	3,245 MiB	1.19×, 1.20×	PASS
keep every byte, stop touching it (pause)	**3,709 MiB**	**1.01×, 1.01×**	**PASS — least disruptive**

- The witness reproduced in **6/6 matched trials across two independent runs** (2.28–2.38×).
- Releasing only one eighth (~460 MiB, just enough to fall below capacity) is enough; the residual 1.19× equals the at-capacity level of Result 1.
- **Pausing activity while keeping all memory** is both the most effective and the least disruptive fix. Across the study, idle/paused co-tenant: 3 valid trials at 1.01–1.03×; active: 6 trials at 2.28–2.38×, same memory held.

14.4 Detector check — does the original SpillGuard detector predict harm?

On the 9 valid research trials the counter-based detector raised SPILL_START on all 4 real failures (**recall 4/4**) but also on 3 of the 5 non-failures (the two at-capacity trials and the idle trial): **precision 4/7**. Confusion matrix: TP 4, FP 3, FN 0, TN 2. This is the measured reason the project moves from "watch counters" to "diagnose by intervention".

14.5 What this does and does not show

SHOWS

- A repeatable, matched, recovered severe slowdown of a real local LLM caused by a controlled co-tenant crossing VRAM capacity (6/6 trials, two runs).
- A measured dose-response cliff.
- A case where counters cannot separate harmful from benign pressure.
- Five owned remedies that restore performance.

DOES NOT SHOW

- The residency mechanism (unverified hypothesis).
- Other GPUs, vendors, drivers, models or engines.
- Behaviour of third-party co-tenants (games, browsers).
- Statistical significance (2 trials per level). It is a **pilot** on one machine.

15. Testing and verification

- **Front end (Vitest)**: 66 tests in 12 files pass — scheduler, kernel dispatch (Round-Robin quantum), memory manager (FIFO/LRU/CLOCK), Banker's, deadlock engine, VFS, advanced subsystems, SpillGuard admission, report parsing, and a test that every built-in research file parses.
- **Agent (pytest)**: 149 tests pass — collector parsing, analyzer/detector on recorded traces, fit estimator, Banker's, server security (token/Origin/Host), recipe validation, evidence classifier, bundle integrity, reducer and remedy workflows.
- **Type checking**: `tsc` with no errors; production build with Vite.
- **In-app**: the Kernel Test Suite window runs its checks live.
- **Bug fixed during the project**: in the dual-core step, a process left in RUNNING was skipped by the scheduler forever, so each process ran one tick and the Round-Robin quantum never applied. Fix: at the start of every tick every RUNNING process returns to READY and competes again; the policy (queue order) keeps it on the CPU until its quantum ends. A test now checks the pattern "A A B B".

16. Limitations and future work

Limitations (state them honestly)

1. The AetherOS kernel is a **simulation** in the browser, not a bootable OS; OPTIMAL replacement falls back to LRU; eBPF scripts are not compiled.
2. One machine, one GPU vendor (NVIDIA), one driver, one target model; small samples (2 trials per level, 9 valid trials for the detector check).
3. The residency/churn mechanism is a hypothesis; PIX/ETW traces are needed.
4. The co-tenant is a synthetic CUDA process, not a real game or browser.
5. The live detector's thresholds were perfect on the synthetic W1 ramp, so W1 does not discriminate between threshold choices; real-world negatives are needed.
6. Windows per-process GPU counters can drift after an app is hidden/restored (a known Microsoft issue); SpillGuard cross-checks with NVML but does not yet flag disagreements.
7. Banker's needs maximum claims; for unknown apps "peak × 1.1" is a heuristic.

Future work

- Run the phase/residency-aware reducer on the real witness (RQ2) and compare with plain ddmin.
- Verify the mechanism with PIX / ETW residency events; add GPU clock/temperature controls.
- Larger samples, randomized order, a second target (another model or engine), real co-tenants, AMD/Intel GPUs.
- Turn "pause the co-tenant" into an automatic, consent-based remedy in SpillGuard.
- Move the AetherOS kernel into a Web Worker; package the agent as an `.exe`.

17. Viva questions and answers

A. About AetherOS

Q. Is AetherOS a real operating system?

A. No. It is a web application that simulates an OS kernel to teach its mechanisms. It runs inside the browser on top of Windows. The scheduling, paging, deadlock and journaling algorithms are real implementations operating on simulated processes, frames and files. The GPU part (SpillGuard) reads the real hardware.

Q. What is a tick?

A. The simulated kernel's unit of time: one call of `Kernel.step()`, every 800 ms by default. One tick = one unit of CPU time for each core.

Q. Why is the kernel deterministic, and why does that matter?

A. It uses no random numbers; the same inputs always give the same schedule. This makes behaviour explainable and lets tests assert exact outputs (e.g. Round Robin gives "A A B B").

Q. Which scheduling algorithm is the default and why?

A. MLFQ, because it approximates SJF without knowing burst times: new/interactive jobs stay in the high-priority short-quantum queue, CPU-bound jobs sink, and aging every 40 ticks prevents starvation.

Q. What is the time quantum and what happens if it is too small or too large?

A. The maximum CPU time a process gets before preemption in Round Robin (4 ticks here; MLFQ uses 4/8/16). Too small: many context switches (overhead). Too large: RR degenerates into FCFS with poor response time.

Q. What is starvation and how do you prevent it?

A. A ready process never gets the CPU because others always win (e.g. long jobs under SJF, low priority under Priority). Prevention: aging — raising the priority of waiting processes; MLFQ's periodic boost.

Q. What is the convoy effect?

A. In FCFS, many short processes wait behind one long CPU-bound process, increasing average waiting time.

Q. Difference between SJF and SRTF?

A. SJF picks the shortest total burst; SRTF picks the shortest remaining time and preempts if a shorter job arrives. SRTF is the preemptive version.

Q. How do you use two cores?

A. Each core has its own run queue. New processes go to the shorter queue (load balancing) unless pinned by affinity. Every tick each core picks its own process; the same process cannot be on both. If one queue is empty and the other has more than one process, the idle core steals half of them from the tail (work stealing).

Q. What is a context switch and how do you count it?

A. Saving one process's CPU state and loading another's. AetherOS counts one whenever a core runs a different PID than in the previous tick, and emits a SCHED_SWITCH eBPF event.

Q. Define turnaround, waiting and response time.

A. Turnaround = completion − arrival. Waiting = time spent ready but not running. Response = first time on CPU − arrival.

Q. How big is AetherOS's memory and how are pages mapped?
A. 32 frames × 4 KB. A process of S KB needs ⌈S/4⌉ pages; free frames are taken first; if there are not enough, the missing ones are page faults and the replacement policy picks victims from other processes.

Q. Explain the CLOCK algorithm.
A. Frames form a circle with a hand. Each frame has a reference bit set on access. To find a victim the hand moves: if the bit is 1 it is cleared (second chance) and the hand moves on; the first frame with bit 0 is evicted. It approximates LRU cheaply; two sweeps always find a victim.

Q. Why doesn't OPTIMAL really work in your OS?
A. Optimal needs the future sequence of page references, which no online system knows. The kernel does not model a future reference string, so it falls back to LRU and the UI says so. Optimal is used as a theoretical benchmark.

Q. What is Belady's anomaly?
A. With FIFO, giving a process more frames can sometimes cause more page faults. LRU and Optimal are stack algorithms and do not have it.

Q. What is thrashing?
A. When processes spend more time moving pages between memory tiers than doing work, because their working sets do not fit. The GPU spill in this project is the same phenomenon one level up: VRAM is too small for the combined working sets.

Q. What happens in your GPF demo?
A. A user-mode (Ring 3) process tries to access address 0x0020 inside kernel space (0x0000–0x3FFF). The memory manager checks ring and address range, traps a General Protection Fault, records a register dump (EIP, ESP, CR3, …) and blocks the process.

Q. How do you detect a deadlock?
A. Build a wait-for graph (edge P → Q if P waits for a lock Q holds) and run DFS with a recursion stack; a back edge means a cycle. With single-instance mutexes a cycle means deadlock.

Q. Difference between deadlock prevention, avoidance and detection?
A. Prevention makes one Coffman condition impossible (e.g. ordered lock acquisition removes circular wait). Avoidance checks each request and only grants it if the state stays safe (Banker's). Detection lets deadlocks happen, finds cycles and recovers (kill/preempt).

Q. Is an unsafe state a deadlock?
A. No. Unsafe means no safe sequence is guaranteed, so a deadlock *may* happen; a safe state guarantees none will.

Q. Walk through your Banker's example.
A. Available 3 3 2. Need = Max − Allocation. P1 (needs 1 2 2) can run → Work 5 3 2; P3 → 7 4 3; P4 → 7 4 5; P0 → 7 5 5; P2 → 10 5 7. Safe sequence P1, P3, P4, P0, P2.

Q. What is write-ahead logging?
A. Recording an operation in a journal before applying it. After a crash, committed journal entries are replayed and uncommitted ones are discarded, so metadata is never left half-updated.

Q. What does fsck do in your OS?
A. After the simulated power cut it scans the journal, rolls back the PENDING (torn) transaction, replays COMMITTED but not yet checkpointed ones, marks the superblock clean and remounts the disk.

Q. What is an inode? A superblock?
A. An inode is the per-file metadata record (type, size, permissions, parent). The superblock is the file-system-wide record (clean/dirty, mounted, counts, block size, journal pointers).

Q. What is eBPF?
A. A Linux technology that runs small verified programs at kernel hooks to count or trace events safely without changing kernel code. AetherOS simulates probes on context switches, page faults and syscalls, with hash maps and an event ring buffer.

B. GPU memory and SpillGuard

Q. What is VRAM?
A. The GPU's own dedicated, very high-bandwidth memory on the graphics card — here 6 GB of GDDR6 (6002 MB reported). Measured write bandwidth to it: 163 GB/s.

Q. What is shared GPU memory?
A. System RAM that Windows lets the GPU use as a second tier, up to half of RAM (7827 MB here). The GPU reaches it across PCIe, measured at 13.2 GB/s — 12.4× slower than VRAM.

Q. What is a spill?
A. When GPU data that should be in VRAM is placed in shared system memory because VRAM is full. Programs keep running but slow down, without any error message.

Q. Who moves the data — the app, the driver or Windows?
A. On Windows, the video memory manager VidMm (part of the WDDM graphics kernel) manages residency, and the NVIDIA driver's sysmem fallback lets CUDA allocations succeed in system memory instead of failing. The application is not told.

Q. Why can't the program just ask CUDA how much memory is free?
A. We measured it: `cuMemGetInfo` reported 0 MB free from 5376 MB onward while allocations kept succeeding in system memory. CUDA gives no spill signal, so an external observer is needed.

Q. Where do your numbers come from?
A. Windows PDH performance counters ("GPU Process Memory" Dedicated and Shared Usage per process; "GPU Adapter Memory" per adapter), DXGI for physical VRAM and budget, NVML as a cross-check, psutil for RAM/commit, and Ollama's /api/ps for model placement. No admin rights are needed.

Q. Why compare against physical VRAM instead of the DXGI budget?
A. We observed that under pressure the driver fills VRAM up to 5954 of 6002 MB, far past the 5234 MB budget; and the budget is per calling process. Physical capacity is the real limit.

Q. Explain your detector's thresholds.
A. The adapter must be at ≥ 92 % of VRAM, the process's shared usage must exceed its own unpressured baseline by > 128 MB and still be rising, for 2 consecutive 1-second samples. It ends after 3 clear samples (below 85 % or excess ≤ 64 MB). Hysteresis prevents flicker; the baseline prevents false alarms on browsers that always use shared memory. In the pilot, spill raised shared usage 74 → 266 MB in the first second, so 128 MB with k = 2 detects within about 2 s.

Q. How accurate is the detector?
A. On synthetic spill runs: recall 3/3, zero false positives, 0.95 s latency. But on the research trials it flagged 3 harmless cases (precision 4/7) — because counters look identical for harmful and harmless pressure. That limitation motivated the intervention-based approach.

Q. How did you get ground truth for the detector?
A. Independently of the counters: the stress tool measures the write bandwidth of every 256 MB block after warming the GPU; blocks in system memory are about 12× slower, which marks the true onset.

Q. How does Banker's algorithm relate to GPUs?
A. VRAM is the resource. Running processes have Allocation = measured VRAM and Max = peak × 1.1; a new model launch has Max = its estimated need. If no safe state exists, launching it would push someone into spill. Because real apps never "finish", SpillGuard also computes a long-lived variant that reserves their remaining need.

Q. How do you estimate how much VRAM a model needs?
A. M = weights on GPU + KV cache + buffers + overhead, read from the GGUF file header and calibrated on this laptop (overhead 235 MB, +6.1 MB per 1 k context). Error 46 MB on average, versus 902 MB for Ollama's own estimate.

Q. Is the agent secure?
A. It listens only on 127.0.0.1, needs a per-session token sent in the URL fragment, checks Host and Origin against DNS rebinding and other sites, and its API is read-only. It never launches programs from the browser and never changes driver settings.

Q. How much overhead does SpillGuard add?
A. About 2.3 % of one CPU core (≈ 0.14 % of the 16-thread CPU) and 52 MB of RAM at one sample per second.

C. About the research

Q. What exactly did you prove?
A. That on this laptop, a real local LLM (Qwen2.5-3B in Ollama) decodes about 2.35× slower when a co-tenant process pushes total GPU memory demand 256 MiB past physical VRAM; that it reproduced in 6 of 6 matched trials in two independent runs and fully recovered when the pressure was removed; that the slowdown is a cliff at capacity; and that pausing the co-tenant's activity fixes it while keeping its memory.

Q. Why baseline, pressure and recovery?
A. It is a matched controlled experiment: the same work measured with and without the single change. Recovery confirms causation — if performance returns when pressure is removed, the slowdown was due to the pressure, not heat, background updates or a broken run.

Q. Why is the threshold 2×?
A. It was declared before running, as an operational definition of "severe". It is not a hardware constant. Our results are far from the boundary on both sides (≤ 1.22× vs ≥ 2.28×), so the conclusion does not depend on the exact choice.

Q. Couldn't the slowdown just be the co-tenant using memory bandwidth?
A. The −512 MiB control has the same kind of active co-tenant rewriting ~2.9 GiB every 250 ms, but everything fits: only 1.16×. And the idle co-tenant at +256 MiB holding the same bytes gave 1.03×. Only the combination over-capacity + active produces 2.3×.

Q. Why is the output required to be identical?
A. To prove the work was the same. With temperature 0 and a fixed seed the model produces the same tokens; if the output changed, the trial would measure different work and is marked INCONCLUSIVE.

Q. What does INCONCLUSIVE mean? Did you delete those trials?
A. It means the trial could not answer the question validly (e.g. host RAM guard tripped, model partly offloaded to CPU, pressure not observed, output changed). They are never deleted: they stay in the bundle and the denominator, and are reported.

Q. Why did some trials stop with the commit guard?
A. The laptop has 16 GB of RAM and the co-tenant plus the model plus the browser pushed Windows' commit charge to 90 %. A trial under host-RAM pressure would mix a different cause (type C) into the result, so the runner stops and marks it INCONCLUSIVE. The guard was never relaxed; memory was freed by closing apps.

Q. Why is the slowdown about 2.35× and not 12× like the bandwidth test?
A. The bandwidth test measured blocks that were completely in system memory. In the LLM case only a part of the working data is affected and the GPU still does most of its work from VRAM, so the end-to-end per-token time rises less than the raw bandwidth ratio.

Q. Why does +512 MiB not make it worse than +256 MiB?
A. Observed, not yet explained — a plateau. A plausible explanation is that once over capacity, the manager settles into the same pattern of moving data regardless of the exact excess; verifying it needs residency tracing (future work).

Q. Is two trials per level enough?
A. For a pilot, it shows repeatability (6/6 FAIL trials across two runs, all very consistent: 2.28–2.38×), but not statistical significance. Larger samples are listed as future work.

Q. What is a SHA-256 manifest for?
A. It stores the hash of every file in a bundle, so any later change to the data is detected by `verify`. It proves integrity, not scientific correctness.

Q. What is ddmin and 1-minimal?
A. Delta debugging minimization repeatedly deletes parts of a failing input and keeps deletions that still fail. 1-minimal means removing any single remaining part makes the failure disappear. It is implemented and tested on a fixture; applying it to the real witness is next.

Q. What is new compared to existing tools?
A. Existing tools (Task Manager, nvidia-smi, other spill trackers) show counters. This project (1) detects spill per process with a baseline and culprit, (2) shows with a measured example that counters cannot separate harmful from harmless pressure, and (3) diagnoses by controlled intervention with matched, recovered, tamper-evident trials and compares remedies.

Q. What would you do next?
A. Run the reducer on the real witness, verify the mechanism with ETW/PIX residency events, add more trials, a second model or engine, real co-tenants and other GPUs, and make "pause the co-tenant" an automatic, consent-based remedy.

18. Glossary A–Z

Adapter
Windows' name for a graphics device (GPU). Identified by a LUID.

Affinity (core affinity)
A restriction on which CPU core a process may run on.

Aging
Gradually raising the priority of waiting processes to prevent starvation (MLFQ boost every 40 ticks).

Allocation matrix
Table of how many instances of each resource each process currently holds.

Back edge
In DFS, an edge to a node still on the recursion stack; indicates a cycle.

Bandwidth
Data moved per second (GB/s).

Banker's algorithm
Dijkstra's deadlock-avoidance algorithm: grant requests only if a safe sequence still exists.

Baseline (detector)
A process's minimum shared GPU usage while the adapter was not under pressure.

Baseline (experiment)
The first stage of a matched trial: the target alone.

Belady's anomaly
FIFO's property that more frames can cause more faults.

Budget (DXGI)
VRAM amount Windows suggests the calling process use; per process.

Burst time
CPU time a process needs.

Bundle
Folder of one experiment's recipe, reports, logs and traces with a SHA-256 manifest.

Checkpoint
Applying a committed journal entry to the main data structures.

CLOCK
Second-chance page replacement using a circular hand and reference bits.

Co-tenant
Another process sharing the same GPU; in the research, an owned CUDA program holding VRAM.

Coffman conditions
Mutual exclusion, hold and wait, no preemption, circular wait — all needed for deadlock.

Cold preparation
The co-tenant becomes resident before the model loads.

Commit charge
Total virtual memory Windows has promised (RAM + page file). Guard trips at 90 %.

Context length (num_ctx)
Number of tokens an LLM can attend to; drives KV-cache size.

Context switch
Saving one process's CPU state and loading another's.

CR3 / EIP / ESP / EAX / CS / EFLAGS
x86 registers: page-table base, instruction pointer, stack pointer, accumulator, code segment, status flags.

CUDA
NVIDIA's platform for general computing on GPUs. The driver API lives in `nvcuda.dll`.

cuMemGetInfo
CUDA call returning free/total GPU memory; reports 0 free during spill, gives no spill signal.

Culprit
The process whose VRAM grew most in the 30 s before a spill.

ddmin
Delta-debugging minimization; produces a 1-minimal failing input.

Deadlock
Processes waiting on each other in a cycle, none able to proceed.

Decode / prefill
Generating output tokens one by one / processing the prompt.

Dedicated Usage
PDH counter: a process's GPU bytes resident in VRAM.

DFS
Depth-first search graph traversal.

Dose-response
Measuring an effect at several strengths of a cause (here: overflow amounts).

DNS rebinding
Attack where a website's domain is re-pointed to 127.0.0.1 to reach local services; blocked by Host checks.

DXGI
DirectX Graphics Infrastructure; Windows API for adapters and video memory info.

dwm.exe
Desktop Window Manager, Windows' compositor; always uses some GPU memory.

eBPF
Extended Berkeley Packet Filter; safe programs attached to kernel hooks.

Evict / demote
Move data from the fast tier (VRAM) to the slow tier (system memory).

FAIL / PASS / INCONCLUSIVE
Trial verdicts: slowdown ≥ 2× with recovery / valid trial below 2× / not valid to decide.

FastAPI
Python web framework used for the agent's server.

FCFS
First-Come First-Served scheduling.

FIFO
First-In First-Out page replacement.

Flash attention
A memory-efficient attention algorithm that avoids a large scratch buffer.

Frame / page
Fixed-size unit of physical / virtual memory (4 KB in AetherOS).

fsck
File-system check and repair.

Gantt chart
Timeline showing which process ran on which core at each tick.

GDDR6
Graphics DDR 6, high-bandwidth memory used as VRAM.

GGUF
llama.cpp's model file format with a tensor header.

GPF (#GP)
General Protection Fault: CPU trap on an illegal memory access or privilege violation.

GPU
Graphics Processing Unit; massively parallel processor.

Hysteresis
Different thresholds for entering and leaving a state to avoid flickering.

Inode
Per-file metadata record.

Intervention
Changing one factor on purpose and re-measuring, to establish cause.

JSONL
JSON Lines: one JSON object per line; format of traces.

KV cache
Stored attention keys and values of earlier tokens in an LLM.

Latency (detector)
Time from real spill onset to the alarm (0.95 s).

LLM
Large language model.

Long-lived Banker's
Variant that reserves running processes' remaining need because they never finish.

LRU
Least Recently Used page replacement.

LUID
Locally Unique Identifier of a graphics adapter.

MAE
Mean absolute error.

Matched trial / block
Baseline, pressure and recovery of identical work; a block is a fixed number of such trials.

MiB / MB
2^{20} bytes / 10^{6} bytes.

MLFQ
Multi-Level Feedback Queue scheduling (quanta 4/8/16 here).

Mutex / semaphore
Lock with one holder / counter allowing N holders.

Need matrix
Max − Allocation in Banker's algorithm.

NVML
NVIDIA Management Library (used by nvidia-smi).

Ollama / llama-server.exe / llama.cpp
Local LLM server / its worker process / the C++ engine it uses.

Offload (GPU layers)
Putting model layers on the GPU; partial offload = some stay on CPU (type A).

OPTIMAL
Bélády's replacement: evict the page used farthest in the future.

Origin header
Browser header naming the site that made a request; checked by the agent.

PCB
Process Control Block.

PCIe
PCI Express bus between GPU and CPU/RAM; the slow path during spill.

PDH
Performance Data Helper, the Windows performance-counter API.

Precision / recall
Correct alarms ÷ all alarms / detected cases ÷ all real cases.

Preemption
Taking the CPU (or a resource) away from a process.

Quantization
Storing model numbers with fewer bits (f16, q8_0, q4_0).

Quantum
Time slice in Round Robin / MLFQ.

Recipe
Validated JSON specification of an experiment.

Recovery ratio
Recovery ÷ baseline; must be ≤ 1.25 for a valid FAIL.

Replay mode
Playing a saved trace through the same UI without the agent.

Residency
Whether a GPU allocation currently lives in VRAM.

Ring 0 / Ring 3
Kernel mode / user mode privilege levels.

Round Robin
Time-sliced cyclic scheduling.

Safe state / safe sequence
A state with an order in which all processes can finish / that order.

SHA-256
Cryptographic hash function used for integrity manifests.

Shared Usage / Shared GPU memory
GPU data held in system RAM / the system-RAM tier (max half of RAM).

SJF / SRTF
Shortest Job First / Shortest Remaining Time First.

Slowdown
Pressure decode ms/token ÷ baseline decode ms/token.

SMP
Symmetric Multi-Processing (multiple identical cores).

Spill
GPU data placed in system memory because VRAM is full.

SPILL_START / SPILL_END / OFFLOAD_SPLIT
SpillGuard events: spill began / ended / Ollama split model to CPU.

Starvation
A ready process never being scheduled.

Superblock
File-system-wide metadata (clean, mounted, counts, journal pointers).

Sysmem fallback
NVIDIA driver (536.40+) behaviour: use system memory instead of failing when VRAM is full.

Thrashing
Excessive paging that makes useful work collapse.

Tick
One step of the simulated kernel clock (800 ms default).

Token (LLM)
A unit of text generated by the model.

Token (security)
Random secret proving the browser was opened by the agent.

Turnaround / waiting / response time
Completion – arrival / time ready but not running / first run – arrival.

Type A / B / C
Framework split / driver spill / host RAM pressure.

VidMm
Windows Video Memory Manager inside the WDDM graphics kernel.

Virtual memory
Abstraction letting processes use more memory than is physically present, by moving pages between tiers.

VRAM
The GPU's dedicated memory (6 GB GDDR6 here).

WAL
Write-ahead logging (journaling).

Wait-for graph
Graph with an edge P → Q when P waits for a resource held by Q.

WDDM
Windows Display Driver Model.

Web Worker
A background JavaScript thread in the browser.

WebSocket
A persistent two-way connection between browser and server (`/ws`).

Witness
A saved recipe that reproduces the failure.

Work stealing
An idle core taking queued tasks from a busy core.

Working set
The set of pages a process actively uses.

Zustand
React state-management library used for the stores.

19. Numbers cheat-sheet

Item	Value
Kernel tick	800 ms default
RR quantum · MLFQ quanta · aging	4 ticks · 4 / 8 / 16 · every 40 ticks
Simulated RAM	32 frames × 4 KB; kernel space 0x0000–0x3FFF, user 0x4000–0xFFFF
Banker's demo safe sequence	P1 → P3 → P4 → P0 → P2 (Available 3 3 2)
Journal	25-entry ring buffer; PENDING → COMMITTED → CHECKPOINTED
Physical VRAM · DXGI budget · shared cap	6002 MB · 5234 MB · 7827 MB
VRAM vs spilled bandwidth (E1)	163.3 vs 13.2 GB/s → 12.4× slower; spill from the 23rd 256 MB block
cuMemGetInfo	0 MB free from 5376 MB onward, allocations still succeed
Detector	θhi 0.92, θlo 0.85, Δ 128 MB, k 2, exit 3; recall 3/3, 0 FP, 0.95 s latency (W1)
Fit estimator	O = 235 MB, +6.1 MB / 1 k ctx; MAE 46 MB vs Ollama 902 MB
Agent overhead	2.3 % of one core, 52 MB RAM
Target	qwen2.5:3b-instruct, ctx 2048, 128 tokens, temp 0; footprint ≈ 2421 MiB; baseline ≈ 14 ms/token
Verdict rule	FAIL if slowdown ≥ 2.0 and recovery ≤ 1.25; guards: RAM floor 1.5 GiB, commit 90 %
Dose-response	−512: 1.16× · 0: 1.21× · +256: 2.36× · +512: 2.33×
Witness	+256 MiB reproduced 6/6 (2.28–2.38×); recovery 0.996–1.010×
Idle co-tenant	1.01–1.03× (3 valid trials) with identical counters
Remedies	release all 1.01× · serialize 1.00× · half 1.10× · eighth 1.19× · pause 1.01×
Detector on research trials	TP 4, FP 3, FN 0, TN 2 → recall 4/4, precision 4/7
Tests	66 front-end (Vitest) + 149 agent (pytest), all passing

All measurements: one laptop (RTX 3050 6 GB Laptop GPU, NVIDIA 581.83, Windows 11 26200, AC power), October 2026. Sources in the repository: `docs/SPILLGUARD.md`, `docs/SPILLGUARD_REPRO.md`, `docs/SPILLGUARD_REPRO_RESULTS_20261018.md`, and the evidence bundles under `traces/repro/`.